Assignment 3: Jet Generation Challenge

CDS DS 595 · Spring 2026 · Boston University

Out:	Wednesday, March 4	**Due:**	Wednesday, March 25
Weight:	15% of final grade	**Format:**	Individual

Overview

This assignment is a **competition**. Your goal will be to build the best possible conditional generative model for particle physics jets. You will use **flow matching**—the technique from Lecture 11—applied to point cloud data from the JetNet dataset [1].

Physics motivation. At the Large Hadron Collider, when high-energy particles collide, they produce collimated sprays of particles called **jets**. Different parent particles (gluons, quarks, top quarks, W bosons, Z bosons, ...) produce jets with distinctive internal structure—called *substructure*—that physicists use to identify what created them. Fast, accurate generation of realistic jets is an active area of research in particle physics, where generative models are used as cheap surrogates for expensive Monte Carlo simulations.

You will train a model that generates jets *conditioned on jet type*, and your generated jets will be evaluated against a held-out jet dataset using physics-informed metrics. A starter notebook provides a working but intentionally basic baseline. Your job is to make it better.

Prize. The top 2 submissions on the final leaderboard each receive **2 months of Claude Max 5x** (worth $\sim$ \$200).

Learning objectives. Build a generative model for a new data modality (point clouds); design neural network architectures for set-structured data; learn and use various training tricks to squeeze out performance; use physics-informed metrics to evaluate generative models; and iterate systematically toward better performance.

Background

The data

Each jet is represented as a **point cloud** of up to 30 particles. Each particle has three features:

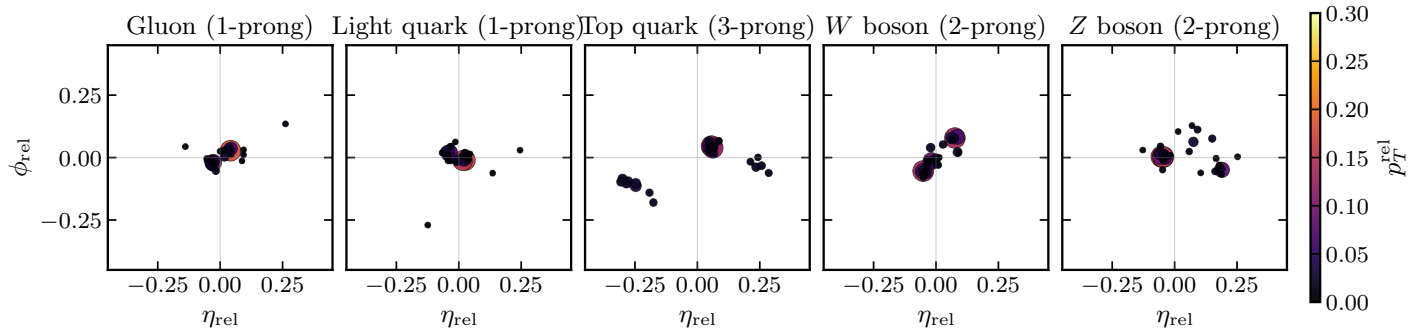
- η_{rel}: relative pseudorapidity (roughly, the angle from the beam axis)
- ϕ_{rel}: relative azimuthal angle (angle around the beam axis)
- p_T^{rel}: relative transverse momentum (fraction of the jet's total p_T)

A fourth feature, the **mask**, indicates which of the 30 particle slots are occupied (1) vs. zero-padded (0). Jets have variable numbers of particles, so the mask is important.

The dataset contains five jet types, each with distinct substructure:

Jet type	Label	Substructure
Gluon	`g`	Diffuse, broad — single-prong
Light quark	`q`	Narrower, fewer particles — single-prong
Top quark	`t`	Three-prong ($t \to Wb \to qqb$)
W boson	`w`	Two-prong ($W \to qq'$)
Z boson	`z`	Two-prong ($Z \to qq$)

When visualized in the (η_{rel}, ϕ_{rel}) plane with marker size proportional to p_T^{rel}, these substructure differences are clearly visible. Top jets show three distinct energy clusters; W/Z jets show two; gluon/quark jets are more diffuse.



Flow matching recap

Recall from lecture: flow matching learns a velocity field $v_\theta(z_t, t)$ that transports samples from a base distribution (noise) to the data distribution. Training uses the **conditional flow matching** loss:

$$\mathcal{L} = \mathbb{E}_{t,\,x,\,\epsilon}\left[\|v_\theta(z_t, t) - (x - \epsilon)\|^2\right], \qquad z_t = (1-t)\,\epsilon + t\,x$$

Generation integrates from $t = 0$ (noise) to $t = 1$ (data) using discretization or an ODE solver. For conditional generation, the velocity network also takes a conditioning signal (here, the jet type).

See the Lecture 11 notes and practical notebook for details.

Evaluation metric

Your generated jets are compared to real jets using the **1-Wasserstein distance** (W1)—a natural metric between probability distributions that measures the minimum "cost" of transforming one distribution into another. For each jet type, we compute W1 on four physics observables: jet mass, and the particle-level marginal distributions of p_T^{rel}, η_{rel}, and ϕ_{rel}. Your **leaderboard score** is the sum of all 20 W1 distances (5 types $\times$ 4 observables). Lower is better.

The data is pre-split into training, validation, and test sets. You receive `train.npz` and `val.npz`; the test set is held out for final scoring. To download the data, run `python download_data.py` from the `code/` directory (see the README for details).

Task

The starter notebook provides a complete but minimal flow matching pipeline for jet generation. Start there.

1. **Understand the baseline.** Run the provided model end-to-end: train, generate samples for each jet type, compute the evaluation metric, and visualize the results. Examine the η-ϕ scatter plots and observable distributions. Where does the baseline fail? Note that the baseline is intentionally bad—it uses a simple MLP that ignores particle structure and has no training tricks.

2. **Improve the overall model.** This is the core of the assignment. Iterate on the model to lower your W1 score—the 1-Wasserstein distance between the distributions of physics observables (jet mass, particle p_T, η, ϕ) computed on your generated jets vs. the real validation data, summed across all 5 jet types. There are many independent axes to explore—you don't need to try all of them, but you should try several and analyze what works.

   Architecture: The baseline uses a flat MLP that ignores particle structure. Consider architectures that process particles individually and aggregate: Deep Sets, graph neural networks, self-attention over particles, etc. Think about what inductive biases matter: particles are a *set* (order shouldn't matter), and the physics is invariant to rotations in ϕ.

   Training tricks: EMA, time sampling schedules, mini-batch optimal transport, informed base distributions, etc.

   Conditioning: The baseline uses simple concatenation. Find and try more sophisticated conditioning strategies for better conditional generation.

3. **Evaluate rigorously.** Use the provided evaluation code to compute your W1 score on the *validation set*. Inspect the per-type, per-observable breakdown to identify weaknesses. Plot observable distributions (generated vs. real) and η-ϕ scatter plots for qualitative assessment.

4. **Submit.** Generate 2,000 jets per type (10,000 total) using your best model. Save them using the provided `save_submission()` function and push to your repository. Your final score will be computed on a **held-out test set** that you do not have access to.

Evaluation

Your generated jets are compared to real jets using the **1-Wasserstein distance** (W1) on physics observables. For each jet type, we compute W1 on:

- Jet mass
- Particle p_T^{rel} (marginal distribution)
- Particle η_{rel} (marginal distribution)
- Particle ϕ_{rel} (marginal distribution)

Your **leaderboard score** is the sum of all W1 distances across the 5 jet types and 4 observables (20 terms total). **Lower is better.**

You can evaluate locally against the validation set at any time using the provided `evaluate.py` script. The final ranking uses a held-out test set with the same metric.

Deliverables

Submit via your GitHub Classroom repository:

1. **Generated jets** (`submission.npz`). A NumPy compressed archive containing 2,000 generated jets per type (10,000 jets total across 5 types). The file must contain 5 jet arrays and 5 corresponding mask arrays:

Key	Shape	Description
`g_jets`, `q_jets`, ..., `z_jets`	`(2000, 30, 3)`	Particle features: $(\eta_{\text{rel}}, \phi_{\text{rel}}, p_T^{\text{rel}})$
`g_masks`, `q_masks`, ..., `z_masks`	`(2000, 30)`	Binary mask (1 = real particle, 0 = padding)

   Rows are jets, columns within each jet are particles (up to 30). Padded particle slots must be zero in both the jets and masks arrays. Helper functions for saving and loading submissions are provided in `utils.py` (`save_submission()`, `load_submission()`).

2. **Generation script** (`generate.py`). A **self-contained** Python script that, when run via `python generate.py`, reproduces your `submission.npz` from scratch. It must:
   - Define your full model architecture (so it can be imported without your notebook)
   - Load your trained parameters from `model_params.pkl`
   - Generate 2,000 jets per type and save `submission.npz`

   A template is provided—update it with your model. **The autograder will run this script to verify your submission.**

3. **Model weights** (`model_params.pkl`). Your trained model parameters, saved via the provided `save_model()` function.

4. **Code**. Your notebook(s) with all training and experimentation code, for reference.

5. **Written report** (`report/report.tex`). Max **3 pages**, NeurIPS format:
   - **Approach** (∼1 page): What did you try and why?
   - **Results and analysis** (∼1 page): W1 score breakdown as a table, observable distribution plots (jet mass, p_T^{rel}, η_{rel}, ϕ_{rel}), η-ϕ visualizations. What worked and what didn't? Why?
   - **AI collaboration** (∼0.25 page max): Which AI tools did you use? How? What strategies were effective? Reflective analysis is best.

Grading

Category	Full marks	Needs improvement
Submission & format (2 pts)	`generate.py` runs and reproduces `submission.npz`; valid format with correct shapes; handily beats baseline W1 score; report is well-formatted, within page limit, clear writing	`generate.py` doesn't run or produces wrong format; does not beat baseline; report is poorly organized, over page limit, or missing sections
Approach (2 pts)	Tried multiple meaningful improvements appropriate for the problem (architecture, training tricks, conditioning); some element of novelty or creative combination; clear rationale for choices	Only ran the baseline or made trivial changes; approaches are not well-motivated; no evidence of systematic experimentation
Analysis (2 pts)	Per-observable W1 breakdown showing understanding of where model succeeds and fails; η-ϕ plots or distribution comparisons; thoughtful discussion of what worked, what didn't, and why	Reports final score without breakdown; no visualizations; no discussion of failures or trade-offs
Leaderboard (2 pts)	Scaled by rank on the held-out test set; top performers get full marks; partial credit proportional to performance	No working submission or no improvement over baseline
AI collaboration (1 pt)	Specific examples of what worked and didn't; insights about effective collaboration; description of models and tools used	Generic description ("I used ChatGPT"); no reflection on the process

Total: 9 points (15% of final grade). Top 2 on the leaderboard additionally receive 2 months of Claude Max 5x.

References

[1] Raghav Kansal, Javier Duarte, Hao Su, Breno Orzari, Thiago Tomei, Maurizio Pierini, Mary Touranakou, Jean-Roch Vlimant, and Dimitrios Gunopulos. Particle cloud generation with message passing generative adversarial networks. *Advances in Neural Information Processing Systems*, 34, 2021. URL https://arxiv.org/abs/2106.11535.